

Ain Shams University

Faculty of Engineering - CSE354 Distributed Computing

Distributed LLM Inference System

Efficient Load Balancing and GPU Cluster Task Distribution for Handling 1000+ Concurrent LLM Requests

Field	Value
Project Type	Distributed Computing Course Project
Main Repository	https://github.com/karimmarwanw/DistributedLLM
Implementation	Python, FastAPI, Ollama, Qdrant, Bash scripts
Prepared For	CSE354 - 2nd Semester 2025/2026
Prepared By	Team 56

This report explains the problem, architecture, implementation, testing, fault tolerance, limitations, and future work for the DistributedLLM project.

1. Executive Summary

DistributedLLM is a distributed LLM serving prototype designed to handle high-concurrency AI requests. The system simulates a real cluster where a client layer submits many requests, a load balancer selects healthy master nodes, master schedulers assign tasks to worker nodes, workers retrieve optional RAG context, and the LLM layer produces a response using either Ollama or a fast simulation path.

The implementation is hosted on GitHub at <https://github.com/karimmarwanw/DistributedLLM>. It focuses on efficient load balancing, GPU cluster task distribution, LLM inference, RAG support, scalability, monitoring, and fault tolerance.

2. Project Requirements Mapping

Required Point	How the Project Handles It
Load balancing	Round Robin, Least Connections, and Load-Aware routing.
GPU cluster task distribution	Two masters distribute work across four worker services representing GPU nodes.
LLM inference handling	Workers call Ollama for real inference or simulation mode for stress tests.
RAG integration	Text/PDF documents are ingested into Qdrant and retrieved as context.
Scalability	The threaded load generator can simulate up to 1000+ concurrent requests.
Fault tolerance	Masters retry failed workers; the load balancer retries failed masters.
Monitoring	Client metrics include latency, throughput, distribution, model usage, and GPU utilization.
Testing	Scripts cover load tests, strategy tests, RAG tests, worker failover, and master failover.

User Story Coverage

User Story	Implemented Support
Send multiple AI requests simultaneously and receive fast responses.	Concurrent client threads and fast simulation mode.
Remain responsive under heavy load.	Load balancing spreads traffic across masters and workers.
Continue functioning if some nodes fail.	Worker and master retry paths preserve availability under partial failure.
Get consistent and accurate LLM responses.	Ollama mode plus RAG context improves answer quality.
Monitor resource utilization.	Health endpoints, logs, and benchmark output expose metrics.

3. System Architecture

The system is split into independent HTTP services. This mirrors a production distributed system while staying practical for a course project. Each service has a narrow responsibility and can run locally or across two physical MacBooks.

Layer	Services / Responsibility
Client	Threaded load generator simulates 100 to 1000+ users.

Layer	Services / Responsibility
Load Balancer :8000	Receives requests and selects Master 0 or Master 1 using Round Robin, Least Connections, or Load-Aware routing.
Master 0 :8001	Schedules Worker 0 :9001 and Worker 1 :9002.
Master 1 :8002	Schedules Worker 2 :9003 and Worker 3 :9004.
Workers	Retrieve RAG context, run Ollama or simulated inference, and return metrics.
RAG API :7100	Stores and retrieves document chunks using Qdrant local vector storage.

Generated Figure 1: Architecture summary with default ports and responsibilities.

4. Component Design

Component	Files	Responsibility
Client	client/load_generator.py	Creates concurrent requests and prints benchmark metrics.
Load Balancer	lb/load_balancer.py	Monitors masters, selects a master, and retries failed masters.
Master Scheduler	master/scheduler.py	Tracks workers, schedules tasks, retries failed workers, and attaches master metrics.
GPU Worker	workers/gpu_worker.py	Retrieves RAG context, runs inference, and reports worker metrics.
LLM Layer	llm/inference.py	Builds prompts and calls Ollama or simulated inference.
RAG Layer	rag/*.py	Ingests documents, chunks text, stores vectors, and retrieves context.

5. Load Balancing and GPU Task Distribution

The load balancer refreshes master health and chooses a master using the selected policy. Round Robin is fair for equal nodes. Least Connections reacts to live load by using active tasks plus in-flight requests. Load-Aware routing adds GPU utilization and request history to avoid overloading a busy master.

- Masters schedule alive workers using round robin.
- Workers expose health endpoints with active task counts and simulated capacity.
- macOS GPU telemetry is sampled through ioreg when available, with simulated fallback metrics.

6. LLM Inference and RAG Integration

Workers query the RAG retriever before inference. Relevant document chunks are added to the prompt and used as the primary source. If no context is found, the LLM still answers from general knowledge or the simulation path. This satisfies the requirement to enhance responses with contextual information.

- PDF ingestion uses pdftotext.
- Text is chunked with overlap.
- Qdrant stores chunk vectors and source metadata.
- Retrieval combines vector search with lexical keyword filtering.
- Ollama mode demonstrates real local LLM inference.

7. Fault Tolerance

Fault tolerance is implemented through health checks, failure marking, and retries. If a worker fails, the master retries another worker. If a master fails or cannot complete a request, the load balancer retries another master. The system remains available as long as at least one valid master-worker path remains alive.

Failure Scenario	Recovery Behavior
Worker fails during processing	Master marks it failed and retries another alive worker.
Master is unreachable	Load balancer marks it down and retries another master.
Master crashes during processing	Load balancer receives an error and reroutes the request.
RAG service unavailable	Worker returns an answer without document context or uses fallback behavior.

8. Testing and Evaluation

Test Area	Example Script	Purpose
Load testing	<code>./scripts/test_load_simulation.sh 1000</code>	Simulate 1000 concurrent requests.
Ollama inference	<code>./scripts/test_ollama_10.sh</code>	Run real local model requests.
RAG retrieval	<code>./scripts/test_rag_retrieval.sh</code>	Verify document retrieval.
Strategy comparison	<code>./scripts/test_strategies.sh 100</code>	Compare routing policies.
Worker failover	<code>./scripts/test_worker_failover_simulation.sh</code>	Verify worker retry behavior.
Master failover	<code>./scripts/test_master_down_failover_ollama.sh</code>	Verify load balancer retry behavior.

Evaluation focuses on success rate, failure count, throughput, latency, master distribution, worker distribution, model distribution, and GPU utilization.

9. Limitations and Future Work

Current limitations:

- Workers represent GPU nodes but do not directly implement CUDA, NVIDIA, or Metal kernels.
- macOS GPU metrics are device-level rather than per-worker-process measurements.
- Simulation mode validates routing scalability but not real LLM throughput.
- Local Qdrant storage is suitable for demos but not production deployment.
- Retries are request-level and do not provide durable queues or exactly-once processing.

Future work:

- Add durable queues for stronger crash recovery.
- Add a live dashboard.
- Add automated pytest coverage.
- Add Docker Compose deployment.

- Add authentication and rate limiting.
- Use neural embeddings for stronger semantic retrieval.

10. Conclusion

DistributedLLM meets the main CSE354 project requirements. It demonstrates a working distributed model for high-concurrency LLM requests with load balancing, master-worker scheduling, RAG integration, monitoring, and fault tolerance. The system can run locally for stress testing, use Ollama for real inference, and run across two MacBooks for a physical distributed demo.

References

- CSE354 Distributed Computing project specification, 2nd Semester 2025/2026.
- DistributedLLM GitHub repository: <https://github.com/karimmarwanw/DistributedLLM>
- FastAPI documentation: <https://fastapi.tiangolo.com/>
- Ollama documentation: <https://ollama.com/>
- Qdrant documentation: <https://qdrant.tech/documentation/>